

Parte Teórica

1–[3 valores] Considere o método seguinte:

```
int f(int m[], int p, int l, int h){
    if(l>=h)
        return 0;
    if(p>m[l]&& p>m[h]){
        l=l+1;
        return f(m,p,l,h);
    }
    else
    {
        h=h/2;
        return 1+f(m,p,l,h);
    }
}
```

Indique, justificando, qual é a ordem de complexidade do método no pior caso.

2 – [3 valores] Considere a string “ARRASADOR”. Construa uma árvore de Huffman adequada e reescreva a string com a codificação resultante.

3 – [3 valores] Apresente todos os passos intermédios necessários para converter a expressão:

$$3 * 2 + 1 = 7$$

numa árvore binária. (o operador = tem prioridade inferior aos operadores + e *)

4 – [3 valores] Qual o resultado da introdução da sequência {5,3,6,2,4} numa *splay tree* vazia?

Parte Prática

5 – [2 valores] Construa, utilizando a classe `PriorityQueue` de forma adequada, uma classe `Biblioteca` que disponibiliza um método que recebe uma livro (objecto da classe `Livro`) e outro que devolve o livro com mais páginas recebido até então (e que ainda não foi devolvido). A classe `Livro` (que não precisa de construir nem pode alterar) não é comparável e disponibiliza um método `numeroPaginas()`.

Para responder a a cada uma das alíneas seguintes, acrescente os métodos, variáveis ou classes internas que considerar serem necessárias.

6 – [3 valores] Considere a seguinte classe, que representa uma árvore binária

```
public class BinaryTree<T extends Comparable<? super T>>{

    static class Node<T>{
        T value;
        Node<T> left,right;
    }
}
```

```

Node(T t){value=t;left=right=null;}
}
// ... etc
}

```

Sabendo que é possível inicializar a árvore de indicando um comparador adequado (bem como por defeito – sem indicar qualquer informação):

- Acrescente os construtores necessários.
- Acrescente um método que devolve o número total de nodos.
- Acrescente um método que devolve a profundidade da árvore.
- Acrescente um método que devolve o número total de valores maiores do que um valor recebido por parâmetro.
- Acrescente um método que acrescenta um valor.
- Acrescente um método que roda um nodo, recebido por parâmetro, com o seu descendente direito.

7 – [3 valores] Considere a seguinte classe, que representa uma árvore AVL:

```

public class AVLTree <T extends Comparable<? super T>>{
    //..
    class Node{
        T value;
        int pl,pr; //profundidade esquerda e direita
        Node left,right;

        Node(T value){
            this.value=value;
            pl=pr=0;
            left=right=nullnode;
        }
    }
}

```

- Acrescente um método que devolve a profundidade de um nodo em tempo constante.
- Acrescente um método que roda um nodo, recebido por parâmetro, com o seu descendente direito (*nota: a resposta não é igual à da pergunta 6.d*).
- Acrescente um método que indica, em tempo constante, se a sub-árvore descendente de um determinado nodo recebido por parâmetro está ou não equilibrada.
- Acrescente um método que acrescenta um valor à árvore.